

Algorithm 1: Data Parallelism Baseline

Comparing Energy Costs of Parallel DNN Training Techniques

Note on AI Usage: Revised with AI for coding support (Claude, Anthropic) per CS 4953 LLM use policy.

Note on Data: All experimental results are original, collected from cluster runs on NCSA Delta. Finalized source code, CSV result files, power logs, and performance plots are posted to the course GitHub at github.com/marisamunoz/Green-DNN-Training-UTSA.

1. Problem

The environmental cost of training AI models has become a real issue. Data centers that run these workloads are consuming massive amounts of electricity and water by 2025, estimates put AI's carbon footprint at the equivalent of New York City and its water usage at the scale of global bottled water consumption annually. Most of the research focus goes toward making models more accurate or faster, but very little asks: which training method actually uses less power to get there?

This project tries to answer that question in a concrete and measurable way. I'm comparing three parallel DNN training techniques PyTorch Distributed Data Parallel (DDP), TensorFlow MirroredStrategy, and AxoNN and measuring not just training speed but actual energy consumption. Algorithm 1 establishes the data parallelism baseline across all three frameworks, so that Algorithms 2 and 3 (mixed precision/quantization and ZeRO memory optimization) have something to compare against.

2. Chosen Algorithm / Method

Data parallelism is the most widely used approach for distributed DNN training and the obvious starting point for this project. The core idea is straightforward: every GPU gets a complete copy of the model, but each one processes a different chunk of the training data at the same time. At the end of each step, the gradients from all GPUs get averaged and the weights get updated. That synchronization step is what makes it work, but it also introduces communication overhead that grows as you add more GPUs and that overhead costs energy.

Rather than implementing data parallelism just once, Professor Prasad's feedback pushed me to compare how three different frameworks handle the same underlying idea. Each one uses a different communication strategy and design philosophy:

Method 1a: PyTorch Distributed Data Parallel (DDP)

PyTorch DDP is the standard baseline. Each GPU runs its own training process, and gradients are synchronized using the NCCL all-reduce operation at the end of each backward pass. DDP is smart about overlapping gradient communication with computation, as soon as a layer's gradients are ready, they start being sent while backprop continues on earlier layers. This overlap reduces but doesn't eliminate the communication cost. Implementation uses `torchrun` to launch multiple processes (Li et al., 2020).

Method 1b: TensorFlow MirroredStrategy

TensorFlow's `MirroredStrategy` is the TF equivalent of DDP; it mirrors the model across all available GPUs and uses all-reduce to synchronize gradients. The main difference is that TF handles distribution at the framework level rather than the process level, which makes it easier to set up but gives less control over the underlying communication. `MirroredStrategy` also scales the batch size automatically by the number of GPUs, which means the effective learning rate needs to be tuned accordingly.

Method 1c: AxoNN

AxoNN (Singh & Bhatele, 2022) was developed specifically for extreme-scale training and takes a different approach to communication entirely. Instead of synchronous NCCL all-reduce, AxoNN uses asynchronous MPI-based point-to-point messaging, which lets GPUs keep computing while communication happens in the background. It also offloads data to CPU memory periodically to reduce GPU memory pressure. AxoNN was shown to outperform Megatron-LM and DeepSpeed on multi-billion parameter transformer models. For CIFAR-10/ResNet18 at our scale, it was an open question whether the asynchronous advantage would hold or whether the simpler synchronous approach of DDP would be more efficient.

3. Underlying Communication Pattern

All three methods use some form of gradient averaging across GPUs, but they do it differently:

- PyTorch DDP: NCCL ring all-reduce gradients flow around a ring topology, each GPU sends to its neighbor until all gradients are summed across the group. Synchronous and blocking before the optimizer step.
- TensorFlow `MirroredStrategy`: Also NCCL all-reduce under the hood, but managed by the TF runtime. Similar pattern to DDP but with additional TF-side overhead from the graph execution model.
- AxoNN: Asynchronous MPI point-to-point instead of waiting for a full all-reduce, GPUs send and receive gradients between neighbors without blocking. Computation and communication overlap more aggressively than in DDP.

The all-reduce communication volume per step is approximately proportional to the number of model parameters. For ResNet18, that's roughly 11 million parameters, or about 44MB in FP32. This stays constant per step regardless of GPU count, which means communication becomes a larger fraction of total time as training steps get faster with more GPUs.

4. Data Structures, Datasets, and Hyperparameters

Model: ResNet18, trained from scratch (no pretrained weights), modified for 10-class output to match CIFAR-10.

Dataset: CIFAR-10, 60,000 32x32 color images across 10 categories (50,000 train, 10,000 test). Each GPU receives a non-overlapping shard via DistributedSampler (PyTorch) or automatic sharding (TF/AxoNN).

Hyperparameters:

- Batch size: 128 per GPU
- Learning rate: 0.1 with cosine annealing over 10 epochs
- Optimizer: SGD with momentum 0.9, weight decay $5e-4$
- Epochs: 10 per run
- Precision: FP32 (full precision baseline, mixed precision is Algorithm 2)
- GPU counts tested: 1 GPU (baseline), 2 GPUs (parallel)

5. Read/Write Contention and Synchronization Overheads

In DDP, the main synchronization point is the all-reduce at the end of each backward pass. Every GPU must wait for this to complete before the optimizer updates weights; this is a hard synchronization barrier. DDP mitigates this by bucketing gradients and overlapping communication with computation, but the barrier still exists.

Read/write contention is relatively low in DDP compared to parameter server approaches because there's no central node being written to by all workers simultaneously. All-reduce is symmetric, so no single node is a bottleneck. In AxoNN, the asynchronous model removes the hard barrier entirely but introduces the risk of stale gradients if a GPU finishes a step before receiving all updates; this is the main tradeoff between synchronous and asynchronous approaches.

Memory-wise, each GPU in DDP holds the full model, optimizer states, gradients, and activations, which for ResNet18 is approximately 500MB-1GB depending on batch size and framework overhead. This is well within the memory limits of the cluster GPUs, so memory contention is not expected to be a limiting factor in this experiment.

6. Parallel Time Complexity

For N GPUs, dataset size D , and per-GPU batch size B :

- Computation per step: $O(B \times \text{model_FLOPs})$, constant per GPU regardless of N , since each GPU only sees $1/N$ of the data
- Communication per step: $O(P / \text{bandwidth})$ where $P \approx 44\text{MB}$ for ResNet18, constant regardless of N for ring all-reduce
- Steps per epoch: $O(D / (N \times B))$, decreases linearly with GPU count

- Total time per epoch: $O((D / (N \times B)) \times (t_{\text{compute}} + t_{\text{communicate}}))$

Ideal speedup would be linear in N . In practice, speedup is sublinear because $t_{\text{communicate}}$ stays roughly constant while t_{compute} per step decreases. As N grows, communication becomes a larger fraction of total time and efficiency drops. For ResNet18 at this scale, speedup from 1 to 2 GPUs should be close to 1.6-1.8x based on prior literature.

7. Timing and Experiment Details

Each configuration was run 3 times to average out variance from cluster scheduling and GPU memory state. The following parameters were varied:

- Number of GPUs: 1 vs. 2
- Framework: PyTorch DDP vs. TensorFlow MirroredStrategy vs. AxoNN

Metrics collected per epoch per run:

- Epoch wall-clock time (seconds), measured with Python `time.time()`
- GPU memory allocated (MB), via `torch.cuda.memory_allocated()` for PyTorch, `nvidia-smi` for TF and AxoNN
- Power draw (Watts), via `nvidia-smi` polling every 5 seconds, logged to CSV
- Training loss and test accuracy per epoch

Energy per epoch was estimated as: Energy (Joules) = average power draw (W) $\times$ total training time (s). Power was computed by summing across all active GPUs per timestamp snapshot and averaging over intervals where GPU utilization exceeded 5%, to exclude idle periods. This methodology follows the approach described in the IEEE MICRO 2025 paper on characterizing distributed training efficiency (cited in bibliography). All experiments were run on NVIDIA A40 GPUs on the NCSA Delta supercomputer.

8. Performance Results

All experiments were completed on NVIDIA A40 GPUs on NCSA Delta. Results below reflect averages across 3 independent runs per configuration, with standard deviation reported in parentheses.

Training Time

AxoNN was the fastest framework at both GPU counts, completing 10 epochs in 37.9s (± 0.2 s) on 1 GPU and 26.7s (± 1.5 s) on 2 GPUs. PyTorch DDP required 44.6s (± 2.4 s) on 1 GPU and 32.4s (± 0.2 s) on 2 GPUs, yielding an actual speedup of 1.38x versus the ideal 2.0x the gap reflects all-reduce communication overhead. TensorFlow was substantially slower than both, requiring 217.2s (± 0.1 s) on 1 GPU and 138.8s (± 4.7 s) on 2 GPUs. The extended TF training time is attributable to XLA graph compilation overhead that PyTorch and AxoNN do not incur.

Test Accuracy

PyTorch DDP 1 GPU achieved the highest final accuracy at 78.4% ($\pm 0.4\%$), followed closely by AxoNN 1 GPU at 77.9% ($\pm 0.7\%$). TensorFlow reached 75.7% ($\pm 2.5\%$) on 1 GPU and 73.7% ($\pm 1.7\%$) on 2 GPUs. AxoNN 2 GPU produced the lowest accuracy of any configuration at 68.2% ($\pm 1.0\%$), a notable 9.7 percentage point drop compared to AxoNN 1 GPU. This degradation is consistent with the known behavior of asynchronous gradient communication at small scale stale gradients can slow convergence when the model is small relative to the communication overhead.

Energy Consumption

Energy was estimated as average active power draw multiplied by total training time. AxoNN 1 GPU was the most energy efficient at approximately 3,537J, followed by PyTorch DDP 1 GPU at 4,570J and DDP 2 GPU at 4,879J. A key finding is that PyTorch DDP 2 GPU consumed slightly more total energy than DDP 1 GPU (4,879J vs 4,570J) despite being 27% faster. This occurs because two GPUs drawing power simultaneously offset the time savings adding a GPU reduces wall clock time but not energy. TensorFlow consumed substantially more energy in all configurations, estimated at 44,245J for 1 GPU and 56,558J for 2 GPUs, primarily due to its longer training time rather than higher per-GPU power draw.

9. Conclusions

The results confirm several expectations from the literature while surfacing findings that were not obvious going in.

PyTorch DDP achieved the best accuracy at both GPU counts and maintained competitive energy efficiency, making it the most reliable choice for this scale of experiment. The measured speedup from 1 to 2 GPUs was 1.38x versus the ideal 2.0x, which is consistent with the communication overhead predicted by the complexity analysis in Section 6 and aligns with findings in the IEEE MICRO 2025 efficiency paper.

AxoNN at 1 GPU was competitive with DDP in both speed and accuracy, and achieved the lowest energy consumption of any configuration. However, scaling to 2 GPUs hurt accuracy significantly without a proportional energy benefit. This suggests that AxoNN's asynchronous design, while powerful at extreme scale, introduces gradient staleness at the ResNet18/CIFAR-10 level that outweighs the communication parallelism benefits.

TensorFlow MirroredStrategy was the slowest and most energy intensive framework in every configuration tested. Its graph compilation model imposes a fixed startup cost that is relatively large compared to the short training runs in this experiment. At larger model sizes or longer training runs, TF's optimized execution graph may recover some of this overhead, but at this scale it is not competitive.

The most significant finding is that adding a GPU does not reliably reduce energy consumption. DDP 2 GPU used more total energy than DDP 1 GPU despite finishing faster. This has practical

implications for how training infrastructure should be chosen when energy cost is a constraint faster is not the same as greener, and the tradeoff depends heavily on framework, scale, and workload.

These baseline results establish the comparison point for Algorithms 2 and 3. The next step is to determine whether mixed precision and quantization techniques (Algorithm 2) can reduce energy consumption while maintaining accuracy relative to this FP32 baseline.

References

- Li, S., et al. "PyTorch Distributed: Experiences on Accelerating Data Parallel Training." VLDB, 2020. <https://arxiv.org/abs/2006.15704>
- Singh, S., Bhatele, A. "AxoNN: An Asynchronous, Message-Driven Parallel Framework for Extreme-Scale Deep Learning." IPDPS, 2022. <https://arxiv.org/abs/2110.13005>
- Sergeev, A., Del Balso, M. "Horovod: Fast and Easy Distributed Deep Learning in TensorFlow." arXiv, 2018. <https://arxiv.org/abs/1802.05799>
- "Characterizing the Efficiency of Distributed Training: A Power, Performance, and Thermal Perspective." IEEE MICRO, 2025. <https://dl.acm.org/doi/10.1145/3725843.3756111>
- Mahmud, H. "Converting Autoencoder Based Energy-Efficient and Secure DNN Inference on Edge Devices." PhD Dissertation, UTSA, 2025.
- Rajbhandari, S., et al. "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models." SC'20, 2020. <https://arxiv.org/abs/1910.02054>